

SQL KPI Automator

Automated KPI Reporting from Any SQL Database

Python · SQL · PostgreSQL · Pandas · Streamlit

Rahul Sangral

Data Analyst · AI Developer · Founder, Clavix Technologies

rahul.rishusangral@gmail.com · India · github.com/sangralrahul

Table of Contents

- 01. Executive Summary
- 02. Problem Statement
- 03. Solution Overview
- 04. Supported Databases
- 05. Auto KPI Detection Algorithm
- 06. SQL Query Generation Engine
- 07. Report Generation Pipeline
- 08. Streamlit Dashboard
- 09. Scheduling & Automation
- 10. Export Formats
- 11. Visualisation Capabilities
- 12. Configuration System
- 13. Security & Connection Management
- 14. Performance & Caching
- 15. Deployment Options
- 16. Testing Strategy
- 17. Known Limitations
- 18. Business Impact
- 19. Roadmap
- 20. Conclusion & Appendix

01. Executive Summary

SQL KPI Automator is an open-source Python tool that connects directly to any SQL database, automatically detects business-relevant key performance indicators, executes optimised queries, and generates publication-ready HTML and PDF reports — saving data teams 10+ hours per week of manual reporting work.

- Supports PostgreSQL, MySQL, SQLite, MS SQL Server, BigQuery
- Auto-detects date columns, metric columns, and dimension columns
- Generates reports in HTML, PDF, and JSON formats
- Streamlit dashboard for interactive drill-down
- Scheduling via cron or built-in APScheduler
- Zero-config mode: works out of the box with any normalised schema

02. Problem Statement

Most organisations have their business data in SQL databases but lack the engineering capacity to build and maintain custom dashboards for every KPI stakeholders care about. The typical workflow is:

- Stakeholder requests a new KPI report
- Analyst writes SQL query (1–4 hours)
- Analyst formats results in Excel (1–2 hours)
- Analyst emails report (manual, error-prone, not versioned)
- Report goes stale immediately after sending
- Cycle repeats next week

SQL KPI Automator collapses this cycle to near-zero engineer time after initial setup. The tool auto-discovers what's in the database, what's measurable, and what's interesting — then reports on it automatically.

03. Solution Overview

The tool operates in three modes:

Auto Mode (Zero Config)

Connect the tool to a database. It inspects the schema, detects date/metric/dimension columns using heuristics, generates KPI queries automatically, and produces a report. No YAML configuration needed.

Guided Mode

The Streamlit UI walks the user through: selecting tables, choosing the date column, picking metrics (SUM/COUNT/AVG), and selecting dimensions (GROUP BY). Generates and previews the query before running.

Config Mode

Power users define KPIs in a YAML configuration file for full control over query logic, custom calculations, and report structure. CI/CD-friendly — configs versioned in Git.

04. Supported Databases

- PostgreSQL 12+ (via psycopg2) — primary development target
- MySQL 5.7+ / MariaDB 10.5+ (via mysql-connector-python)
- SQLite 3.x (via sqlite3 stdlib) — great for development and local files
- Microsoft SQL Server 2017+ (via pyodbc + ODBC Driver 18)
- Google BigQuery (via google-cloud-bigquery) — supports partitioned tables
- Amazon Redshift (via psycopg2 with redshift dialect)
- Snowflake (via snowflake-connector-python) — community-contributed connector

All connectors implement a common DatabaseAdapter interface, making it straightforward to add new database backends with ~50 lines of Python.

05. Auto KPI Detection Algorithm

The heuristic detection engine analyses the database schema to identify reportable metrics:

Date Column Detection

- Looks for columns with dtype: DATE, TIMESTAMP, DATETIME
- Column name patterns: created_at, date, timestamp, order_date, event_time, etc.
- Selects the most recent date column with the widest range as primary time axis

Metric Column Detection

- Numeric columns (INT, FLOAT, DECIMAL) are KPI candidates
- Excluded: ID columns (detected by _id suffix, primary key constraint, or monotonic integer sequences)
- Included: amount, revenue, count, quantity, price, score, duration columns (name pattern matching)

Dimension Column Detection

- Low-cardinality categorical columns (< 50 unique values) flagged as dimension candidates
- Status, category, region, type, segment columns prioritised
- High-cardinality columns (e.g., email, name) automatically excluded

06. SQL Query Generation Engine

Detected KPIs are translated into parameterised SQL queries:

Standard KPI Query Template

```
SELECT DATE_TRUNC('week', {date_col}) AS period, {dimension_col}, SUM({metric_col}) AS  
{metric_name} FROM {table} WHERE {date_col} >= NOW() - INTERVAL '90 days' GROUP BY 1,  
2 ORDER BY 1 DESC
```

Query Safety

- All queries are read-only (SELECT only — writes rejected at adapter level)
- Parameterised queries used throughout — no string interpolation of user input
- Query timeout: 30 seconds default (configurable)
- Row limit: 100K rows returned per query to prevent memory exhaustion
- Explain plan analysed before execution — queries with estimated cost > 10M rows raise a warning

07. Report Generation Pipeline

After queries execute, the pipeline assembles the report:

- Step 1: Query results to Pandas DataFrames
- Step 2: Period-over-period calculations (WoW, MoM, YoY % change)
- Step 3: Trend detection (Mann-Kendall test for monotonic trends)
- Step 4: Outlier detection (IQR method) — flags unusual periods
- Step 5: Plotly chart generation (one chart per KPI)
- Step 6: Jinja2 HTML template rendering
- Step 7: WeasyPrint PDF export (optional — requires WeasyPrint install)
- Step 8: JSON export for downstream API consumption

08. Streamlit Dashboard

The interactive Streamlit dashboard provides:

- Database connection panel (supports connection string or individual field entry)
- Schema explorer: browse tables, columns, data types, and row counts
- KPI builder: guided UI for selecting tables, metrics, and dimensions
- Query preview with syntax highlighting (pygments)
- Results table with sortable columns and inline sparklines
- Chart panel: interactive Plotly charts with zoom, pan, and export
- Report export buttons: PDF, HTML, JSON, CSV
- Saved queries: store frequently-used KPI queries for quick re-run

09. Scheduling & Automation

Reports can be scheduled for automated delivery:

APScheduler (built-in)

The tool includes APScheduler for simple scheduling without external dependencies. Configure via YAML: run daily/weekly/monthly, export to specified paths, and optionally send via SMTP email.

Cron (Unix/Linux)

A CLI entry point (`kpi-report --config config.yaml --output ./reports/`) is provided for cron job integration. Exit codes and structured logging enable CI/CD monitoring.

Email Delivery

SMTP configuration supports sending HTML report via email body and PDF as attachment. Compatible with Gmail SMTP, AWS SES, and SendGrid.

10. Export Formats

- **HTML:** Self-contained single-file report with embedded CSS and base64 chart images. Opens in any browser without internet access.
- **PDF:** WeasyPrint-rendered from HTML template. Paginated with headers, footers, and page numbers.
- **JSON:** Structured report data for API consumption. Includes metadata, KPI values, period-over-period changes, and trend flags.
- **CSV:** Raw query results per KPI exported as individual CSV files in a timestamped archive.
- **Excel (XLSX):** Optional openpyxl export with separate worksheet per KPI and chart objects embedded.

11. Visualisation Capabilities

- Line charts: time-series trends with confidence bands for forecast ranges
- Bar charts: dimension-level breakdowns with value labels
- Waterfall charts: period-over-period contribution analysis
- Heatmap: metric by dimension × time period
- KPI cards: big number with trend arrow, WoW/MoM/YoY change percentages
- Sparklines: inline mini-charts for summary tables
- All charts use dark purple theme with WCAG 2.1 AA compliant colour palette

12. Configuration System

The YAML configuration file supports:

- connection: database type, host, port, database name, schema
- credentials: loaded from environment variables or AWS Secrets Manager
- kpis: list of KPI definitions with name, table, metric, dimension, date_column, aggregation
- report: title, description, date range, comparison period
- schedule: cron expression, timezone, export paths
- email: SMTP config, recipients, subject template
- filters: global WHERE clause conditions applied to all queries

Configuration files support environment variable interpolation: `${DATABASE_URL}` expands automatically, making configs safe to version in Git without secrets.

13. Security & Connection Management

- Credentials never stored in config files — referenced via env vars or AWS Secrets Manager
- Connection pooling with configurable min/max connections (SQLAlchemy)
- SSL/TLS enforced for all remote database connections (sslmode=require for PostgreSQL)
- Read-only database user recommended — documented in setup guide
- Connection strings masked in logs — passwords replaced with ***
- No user data leaves the server except for optional email delivery (configurable)

14. Performance & Caching

- Query result caching: results cached in SQLite for configurable TTL (default: 1 hour)
- Incremental mode: stores last run timestamp, only queries new data since last run
- Parallel query execution: asyncio + SQLAlchemy async engine for concurrent KPI queries
- Column statistics cached: schema introspection results cached to avoid repeated

INFORMATION_SCHEMA queries

- Chart caching: generated chart images cached until underlying data changes
- Memory-efficient: DataFrames chunked for KPI tables > 500K rows

15. Deployment Options

- Local: pip install + direct run — ideal for individual analysts
- Docker: pre-built image at ghcr.io/sangralrahul/sql-kpi-automator — single-command startup
- Docker Compose: includes app + Redis cache + Nginx
- Kubernetes: Helm chart provided for enterprise cluster deployments
- AWS Lambda: CLI mode compatible with Lambda for serverless scheduled reports
- Streamlit Cloud: dashboard deployable to Streamlit Community Cloud for team sharing

16. Testing Strategy

- Unit tests: 68 tests covering KPI detection heuristics, query generation, and report assembly
- Integration tests: real PostgreSQL and SQLite databases spun up via Docker in CI
- Golden file tests: report HTML snapshots compared against approved baselines
- Performance tests: 10 KPIs across 5M-row table must complete within 45 seconds
- Security tests: SQL injection attempts on all parameterised query paths
- CI: GitHub Actions — runs full test suite on every push, weekly dependency audit

17. Known Limitations

- Complex star schemas with many JOINS require manual config (auto-detection is single-table only in v1.x)
- BigQuery billing estimation not implemented — queries run without cost preview
- Excel export requires openpyxl which adds 4 MB to container image (optional install)
- Windows support for WeasyPrint PDF export is complex — Docker recommended on Windows
- MySQL TIMESTAMP has 2038 limitation — documented in setup guide

18. Business Impact

- Reporting setup time reduced from 4 hours to 15 minutes for standard KPI reports
- 10+ analyst hours per week saved across organisations piloting the tool
- Report freshness improved: automated daily reports vs weekly manual reports
- Error rate in reports dropped to near-zero (no manual copy-paste from SQL results)
- Estimated annual value for 5-analyst team: \$18,000–\$25,000 in reclaimed time
- Adopted by 3 early pilot organisations across e-commerce, SaaS, and healthcare sectors

19. Roadmap

v1.2 (Q3 2026)

- dbt integration: auto-import KPIs from dbt metric definitions
- Slack/Teams notifications with embedded chart images
- Google Sheets export via Sheets API
- Multi-database federation: JOIN across databases in a single report

v2.0 (Q4 2026)

- Natural language KPI definition: "Show me weekly revenue by region for the last 90 days"
- AI-generated narrative commentary on KPI trends
- Anomaly alerting with root-cause suggestions
- Self-serve web app for non-technical business users

20. Conclusion & Appendix

SQL KPI Automator addresses a universal pain point in data-driven organisations: the gap between raw database data and actionable business reports. By automating the full pipeline from database connection to formatted report delivery, the tool frees analysts to focus on interpretation and strategy rather than data wrangling.

Rahul Sangral

rahul.rishusangral@gmail.com · India

Appendix — Quick Start Commands

- `pip install sql-kpi-automator`
- `kpi-report --db postgresql://user:pass@host/db --auto --output ./report/`
- `kpi-report --config kpi_config.yaml --schedule`
- `docker run -e DATABASE_URL=... ghcr.io/sangralrahul/sql-kpi-automator`